# 02-750 Week 9

Automation of Scientific Research

Aidan Jan

March 24, 2026

## Background: Regression

The goal of regression analysis is to learn a map / function, $f : \mathbb{R}^d \to \mathbb{R}$

- As is the case for learning classifiers, the choice of training instances impacts generalization error, thus, we want to select them actively.

Regression models (like classifiers) can be divided into two categories:

- **Parametric:** the model **summarizes** the training data using a **fixed** number of parameters (independent of the size of the training data)
  - For example, linear regression: $y = x^T\beta$
- **Non-parametric:** the number of parameters is **proportional** to the amount of training data, and the predictions are often made using the actual training instances
  - For example, Gaussian Process Regression, KNN regression, Regression Trees

## Background: Bias-Variance Decomposition

A model's **generalization error** can be decomposed as follows:

$$Err(x) = Bias^2 + Variance + Irreducible\ Error$$

- Bias $:= \mathbb{E}_D[\hat{f}(x; D)] - f(x)$
  - This is the error due to the difference between the 'true' function and the one we select from our chosen hypothesis class (Recall PAC learning)
- Variance $:= \mathbb{E}_D\left[\left(\mathbb{E}_D[hat f(x; D)] - \hat{f}(x; D)\right)^2\right]$
  - This is the error due to the training data being a finite sample of $P(X, Y)$. Different training sets will produce different parameters, creating variance.
- Irreducible Error: This is the error due to any noise in the training data

### The bias-variance tradeoff

- Bias and variance are generally in opposition to one another
  - Complex models tend to have low bias but high variance
  - Simple models (i.e., those with few parameters) tend to have high bias but low variance
- Some learning algorithms intentionally introduce bias in order to decrease variance
  - For example, **Lasso** and **Ridge** regression introduce bias to reduce variance

### Regression Analysis

- The field of Statistics has developed a variety of techniques for learning regression models efficiently (i.e., with low sample complexity)
- Examples:
  - Latin Hypercube Sampling
  - Randomized Block Designs
  - Taguchi Methods
  - Optimal Design of Experiments (DOE)

## Optimal Design of Experiments

A branch of statistics devoted to pre-selecting sets of **experimental conditions** that are collectively optimal **with respect to some criterion**

- DOE shares many of the same objectives as Active Learning but...
  - Most DOE techniques are specifically for learning **parametric** regression models
  - Most DOE methods select **all** the experiments *a priori*, not iteratively*
  - **Some DOE methods are more than 100 years old!**

DOE methods often make (strong) assumptions about:

- The **parametric form** of the model (e.g., linear)
- The nature of the noise
- The test density, $P(X)$

### Example: DOE for Linear Regression

Model (in matrix form for $k$ training instances): $\mathbf{y} = X\beta + \epsilon$

- $\mathbf{y}$ is a $k \times 1$ vector containing $k \geq 1$ observations / samples
- $\mathbf{X}$ is the $k \times b$ **design matrix**. Here, $b$ is the number of parameters.
- $\beta$ is the $b \times 1$ parameter vector
- $\epsilon$ is a $k \times 1$ noise / error vector
  - Assumptions
    * $\mathbb{E}[\epsilon_i] = 0$. (Expected value of noise is zero)
    * $Var(\epsilon) = \sigma^2 < \infty$ (Noise is constant and has finite variance)
    * $\mathbb{E}[\epsilon_i \epsilon_j] = 0$ (Noise is uncorrelated across instances)

The standard way of estimating $\beta$ via **ordinary least squares** (OLS)

- Given a vector of $k \geq b$ observations / samples the OLS estimate is:

$$\hat{\beta} = (X'X)^{-1}X'y$$

  - Once we estimate $\hat{\beta}$, the predicted value for any $x$ is: $\hat{y} = x\hat{\beta}$
- It can be shown (via the Gauss-Markov Theorem) that:

- OLS produces a $\hat{\beta}$ that minimizes $\sum_{i=1}^{k}(\hat{y}_i - y_i)^2$
- $\mathbb{E}[\hat{\beta} - \beta_{true}] = 0$ (i.e., OLS is an unbiased estimator)
- $Var(\hat{\beta}) = \sigma^2(X'X)^{-1}$
- $Var(\hat{y}) = \sigma^2 x(X'X)^{-1}x'$

Recall the bias-variance decomposition:

$$Err(x) = Bias^2 + Variance + Irreducible\ Error$$

- OLS is unbiased, so $Err(x) = Variance + Irreducible\ Error$
- We know that $Var(\hat{\beta}) = \sigma^2(X'X)^{-1}$
  - $\sigma^2$ is usually unknown, but it is a constant, so we can ignore
  - So the **design matrix**, $X$ is really the **only thing that matters**, not the $y$'s.

DOE methods seek to minimize the variance in the estimate (and thus generalization error) by selecting an optimal set of $k$ training instances (i.e., rows of $X$)

- Notice that these instances can be selected without knowing **any** labels!
- Different DOE methods use different definitions for what it means to be optimal

## Quantifying Uncertainty for Linear Regression

- Given $D_L = \{(x_1, y_1), \cdots, (x_n, y_n)\}$, the sum of squared vertical deviations from the point to the line is

$$f(b_0, b_1) = \sum_{i=1}^{n}[y_i - (b_0 + b_1 x_i)]^2$$

- The point estimates of $\beta_0$ and $\beta_1$, denoted by $\hat{\beta}_0$ and $\hat{\beta}_1$, are the **least squares estimates**, that is, the values that minimize $f(b_0, b_1)$

$$\begin{aligned}
b_1 = \hat{\beta}_1 &= \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{s_{xy}}{s_{xx}} \\
b_0 = \hat{\beta}_0 &= \frac{\sum_{i=1}^{n} y_i - \hat{\beta}_1 \sum_{i=1}^{n} x_i}{n} = \bar{y} - \hat{\beta}_1 \bar{x} \\
s_{xy} &= \sum_{i=1}^{n} x_i y_i - \frac{\sum_{i=1}^{n} x_i \sum_{i=1}^{n} y_i}{n} \\
s_{xx} &= \sum_{i=1}^{n} x_i^2 - \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n} \\
\hat{\sigma}^2 = s^2 &= \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n-2} = \frac{1}{n-2}\sum_{i=1}^{n}\hat{e}_i^2
\end{aligned}$$

## Optimality Criteria: G-Optimality

$$\min_{x_i, i=1:k} \max_{x \in \mathcal{X}} \mathrm{Var}(\hat{y}_x)$$

- This criterion selects the $k$ instances that minimize the maximum variance of any prediction
  - This is equivalent to minimizing the maximum element in the diagonal matrix $X(X'X)^{-1}X'$
- This is the criterion proposed by Smith

## Optimality Criteria: D-Optimality

$$\max_{x_i, i=1:k} |X'X|$$

- Select the $k$ instances that maximize the *determinant* of $X'X$
- Geometric Interpretation
  - $|X'X|$ is inversely proportional to the product of the axes of the **confidence ellipsoid** around the point estimate of our parameter vector $\beta$
  - Bigger $|X'X|$ means smaller ellipsoid, thus, higher confidence in $\beta$

## Optimality Criteria: A-Optimality and E-Optimality

**A-Optimality:** (Chernoff 1953)

$$\min_{x_i, i=1:k} \mathrm{Tr}\left((X'X)^{-1}\right)$$

- Select the $k$ instances that minimize the *trace* of the inverse $X'X$
- Minimizes the average variance of the parameter estimates

**E-Optimality:** (Ehrenfeld 1953)

- Select the $k$ instances that maximizes the minimum eigenvalue of $X'X$
- Minimizes the maximum variance of $\beta$

## DOE for Nonlinear Models

If the "true" model is non-linear, but can be expressed as a linear **basis function**, we can still use the previous objectives to find optimal designs

- A linear basis function has the form: $f(x) = \sum_{i=1}^{b} \alpha_i \phi_i(x)$
  - Here, each $\phi_i$ is one of the basis functions
- Common Basis Functions: Polynomial, Fourier, Gaussian, Sigmoid

**Example:**

Consider the polynomial $f(x) = \alpha_1 c + \alpha_2 x + \alpha_3 x^3 + \alpha_4 x^3$

- The basis functions are $\phi_1 = c, \phi_2 = x, \phi_3 = x^3, \phi_4 = x^3$
- We could construct a $k \times 4$ design matrix, where the points are selected using any of the previously defined criteria (e.g., G-optimality)

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ 1 & x_3 & x_3^2 & x_3^3 \\ 1 & x_4 & x_4^2 & x_4^3 \end{bmatrix}$$

- After obtaining labels, we can use OLS to estimate $\alpha = [\alpha_1 \alpha_2 \alpha_3 \alpha_4]'$

### Summary: Optimal Design of Experiments

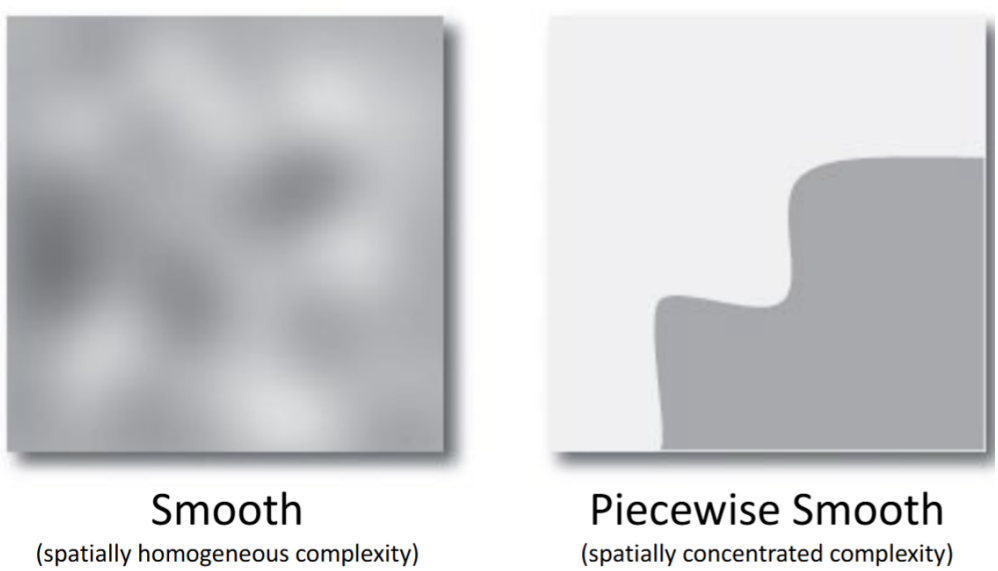
- DOE is primarily used for learning parametric regression models that can be expressed as linear basis functions
- DOE is similar to Active Learning in the sense that it selects training data
  - DOE methods select instances by optimizing some scalar property of $X'X$ which will, in turn, minimized the (expected) generalization error
  - Unlike active learning, DOE methods select the design points at the same time

## Recursive Dyadic Partition (RDP) Algorithm

- RDP is an algorithm for learning **non-parametric regression models**
- Type II (cluster expoiltation). However, unlike ZLG, DH, and PLAL, it is intended for the **membership query synthesis** data access model.
  - Pool-based sampling is possible, if certain conditions are met
- It is **aggressive**
- Guarantees:
  - RDP will converge to the optimal model
  - The upper and lower bounds on the rate of convergence are known

### Rationale

RDP is intended for learning **piecewise smooth functions**.



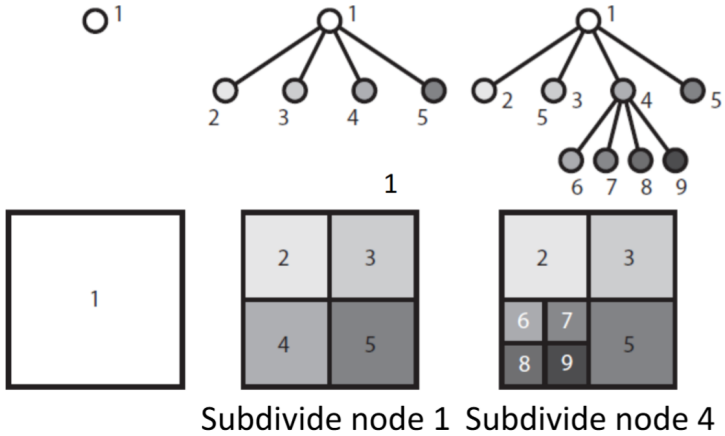

For functions whose complexity is spatially non-uniform and highly concentrated in small subsets of the domain, **the extra spatial adaptivity of the active learning paradigm can lead into significant performance gains**

- The authors show that **there is no benefit to active learning** (relative to passive learning), if the function satisfies the **Hölder condition**.

## RDP: Dyadic Space-Partitioning Trees

The algorithm uses dyadic space-partitioning trees.

- Example in a 2D domain ($2^d$-ary tree structure)




- Consider $f : [0,1]^d \to \mathbb{R}$
  - Step 1: Divide $[0,1]^d$ into $2^d$ equal sized hypercubes
  - Step 2: Repeat this process again on each hypercube
- Repeating this process $\log(m)$ times gives rise to a partition of the unit cube into $m^d$ hypercubes of identical size. (This process is represented as a $2^d$-ary tree structure above.)

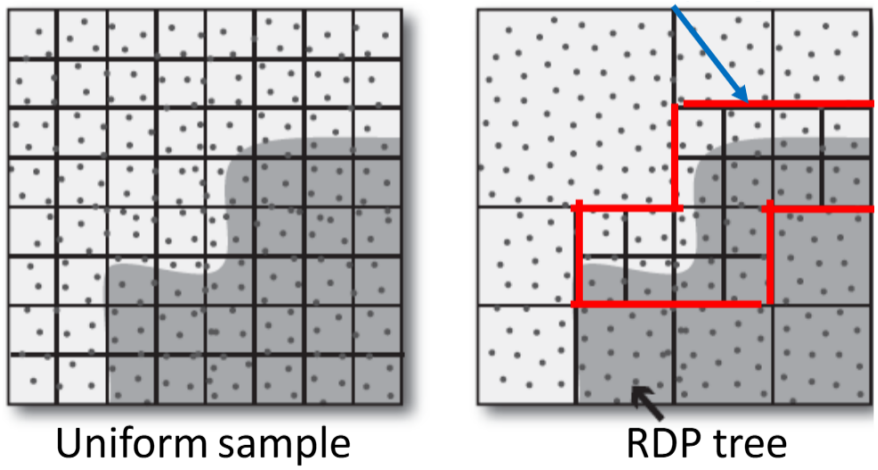
Note: Dyadic means to divide the domain by $2^d$, where $d$ is the dimension of $\mathcal{X} = [0,1]^d$

There are **two phases** for this algorithm.

1. **Preview Phase:**
   - Use half of the labeling budget to label points **uniformly** within the domain, $\mathcal{X}$
   - Learn a bounded height regression tree modified to partition the domain via a **Recursive Dyadic Partition** (RDP)
   - Use the RDP model to identify boundaries in the domain
2. **Refinement Phase:**
   - Use the remaining half of the budget to label the boundary regions densely
   - Learn a regression model from the labeled points

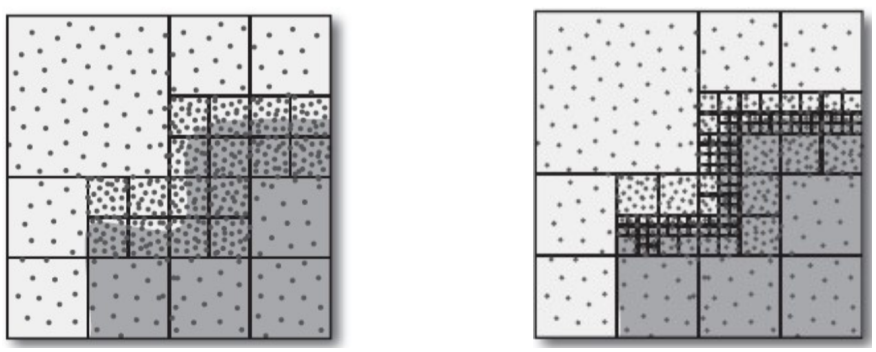
**Example:**

**Preview Phase:**




The deepest nodes in the tree will tend to be in boundary regions (i.e., places where the function changes quickly)

**Refinement Phase:**



Use the remaining budget to desnsely sample the regions of the domain corresponding to the deepest nodes of the regression tree

**Technical Issue**

- If the dyadic partition is aligned with a boundary, the refinement phase is unlikely to sample in that region
- This can be addressed by building multiple RDP models, shifted along one or more dimensions. For example, the bottom left corner in the above examples.

## RDP Guarantees

**Consistency and Rate of Convergence** (under certain assumptions on $f$)

- The authors derive upper and lower bounds for the rate of convergence to the optimal model
  - Upper bound: $\mathbb{E}\left[\|\hat{f} - f\|^2\right] \leq C\left(\frac{n}{\log n}\right)^{\frac{1}{d-1+\frac{1}{d}}}$
  - Lower bound: $\mathbb{E}\left[\|\hat{f} - f\|^2\right] \leq cn^{-\frac{1}{d-1}}$
- The corresponding bounds for passive learning are as follows:
  - Upper bound: $\mathbb{E}\left[\|\hat{f} - f\|^2\right] \leq C\left(\frac{n}{\log n}\right)^{-\frac{1}{d}}$
  - Lower bound: $\mathbb{E}\left[\|\hat{f} - f\|^2\right] \leq cn^{-\frac{1}{d}}$

- That is, RDP's complexity is essentially that of passive learning, but on a lower-dimensional space

### RDP Algorithm: Summary

- The algorithm is elegant and easy to implement, but **the number of samples needed to obtain reasonable accuracies will be large in practice**, since we need to guarantee sufficient uniform sampling in a $d$-dimensional space, which is expensive (see *curse of dimensionality*)
- If desired, we can use the labeled examples to train a more sophisticated regression model
- The algorithm is noteworthy because it is **aggressive**, yet the authors are able to prove theorems about learning rates
  - Note, however, that these guarantees only hold if the underlying function is piecewise continuous

### Summary: DOE vs. Active Learning

Objective: Generate dataset from which one can learn an accurate predictive model

| Category | Design of Experiments | Active Learning |
|---|---|---|
| Logistics | Limited to one batch (e.g., clinical trials) | Multiple batches (e.g., compound screening) |
| Feature knowledge | Good when you know all your features are useful | Good when you don't know if your features are useful |
| Model assumptions | Typically assumes data are derived from a specific model | No assumptions needed |

## ALICE

**ALICE:** "Active Learning in Approximately Linear Regression Based on Conditional Expectation of Generalization Error", Sugiyama (JMLR, 2006)

### Aleatoric Uncertainty vs. Epistemic Uncertainty

- **Aleatoric uncertainty:** also known as data uncertainty, is irreducible uncertainty in data that gives rise to uncertainty in predictions
  - This type of uncertainty is not a property of the model, but rather is an inherent property of the data distribution, and hence, it is irreducible
- **Epistemic uncertainty:** also known as knowledge uncertainty, occurs due to inadequate knowledge
  - One can define models to answer different questions in model-based prediction

### The ALICE Algorithm

**Key Features:**

- ALICE is an algorithm for learning parametric regression models that take the form of **linear basis functions**
- Unlike most query selection methods, **it is not iterative!**
  - It constructs an A-optimal design, and then queries those points all at once
  - It is intended for the membership query synthesis data access model
    - Pool-based sampling is possible, if certain conditions are met

- It is **aggressive**, but corrects for sampling bias using **importance weighting** (similar to IWAL)
- Guarantees
  - ALICE will converge to the optimal model
  - Under the assumption that the true model is approximately linear, the author proves bounds on the learning rate

## ALICE: Motivation

Traditional DOE methods **assume that we know the form of the true function**

- Suppose that we are wrong. How bad could the generalization error be?
- For example:
  - True function: $f(x) = 1 - x + x^2 + \delta x^3 + \epsilon$
  - Assume that $\epsilon \sim \mathcal{N}(0, 0.3)$
  - Our assumed model: $g(x) = \alpha_1 - \alpha_2 x + \alpha_3 x^2$
  - Compare A-optimal design vs a random selection of $k = 10$ labeled instances

| Generalization Error (k = 10) | $\delta = 0$ | $\delta = 0.05$ | $\delta = 0.5$ |
|---|---|---|---|
| A-optimal design: $\min \mathrm{Tr}\left((X'X)^{-1}\right)$ | 1.1 | 2.0 | 95.5 |
| Random Selection | 2.2 | 2.3 | 7.1 |

- The magnitude of the error for $\delta = 0.5$ is due in part to the bias we introduce by misspecifying the model

## ALICE: Background

It can be shown that for any linear estimator that:

$$Variance = \sigma^2 \mathrm{Tr}(ULL') \propto \mathrm{Tr}(ULL')$$

where $L = (X'X)^{-1}X'$ is the **Moore-Penrose pesudoinverse** and

$$U_{i,j} = \int \phi_i(x)\phi_i(x)P_{\text{test}}(x)\,\mathrm{d}x$$

is a $b \times b$ matrix, where $b$ is the number of terms in the linear basis function

- Recall that **for a correctly specified model**, ordinary least squares (OLS) is an unbiased estimator, thus, **minimizing Tr**$(ULL')$ **will minimize the (expected) generalization error**.

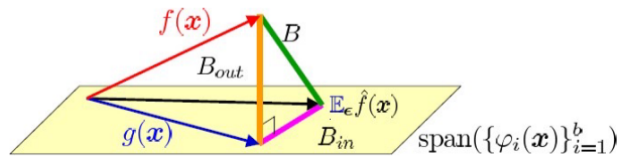
The author decomposes bias into two orthogonal components:

- The **out-of-model bias** due to misspecifying the model;

$$B_{out} = \int \left(f(x) - g(x)\right)^2 P_{\text{test}}(x)\,\mathrm{d}x$$

- The **in-model bias** due to differences in $P_{\text{train}}$ and $P_{\text{test}}$

$$B_{in} = \int \left(\mathbb{E}_{\text{train}}[\hat{f}(x)] - g(x)\right)^2 P_{\text{test}(x)\,\mathrm{d}x}$$




- If we have misspecified the model, $B_{out} > 0$
  - But, we can't reduce it either (unless we are **told** the true form of the function)
    - That is, $B_{out}$ is a kind of irreducible error
  - So, we can just ignore $B_{out}$ when it comes to selecting design points
- OLS is an unbiased estimator, so is it reasonable to assume that $B_{in} = 0$?
  - Unfortunately, OLS is only unbiased if $P_{train} = P_{test}$, which isn't true in Active Learning, so $B_{in} > 0$
  - Solution: Use **Importance Weighting** (just like the IWAL algorithm)

ALICE's overall strategy is to use importance weighting to reduce $B_{in}$, and then to use an A-optimal design criterion to reduce variance, and thus the generalization error

## Recall: Importance Weighting

- The empirical average loss for model $h$ on $n$ instances is: $\frac{1}{n}\sum_i loss(h(x_i), y_i)$
- As $n$ goes to $\infty$ the training loss converges to

$$\int loss(h(x), y) P_{train}(x)\, \mathrm{d}x$$

- Of course, what we really care about is the **test** loss:

$$\int loss(h(x), y) P_{test}(x)\, \mathrm{d}x$$

  - If $P_{train} = P_{test}$, then these losses will be the same
  - But if we have a biased training data, then $P_{train}(\mathcal{X}) \neq P_{test}(\mathcal{X})$
- Recall that many learning algorithms can use **weighted loss functions**
  - Ex. weighted empirical loss: $\widehat{err}(h) = \frac{1}{n}\sum_i w_i loss(h(x_i), y_i)$
- Let the **importance weight** of a training instance be defined as:

$$w_i = \frac{P_{test}(x_i)}{P_{train}(x_i)}$$

- The importance-weighted loss is

$$\widehat{err}_{IW}(h) = \frac{1}{n}\sum_i \frac{P_{test}(x_i)}{P_{train}(x_i)} loss(h(x_i), y_i)$$

- As $n$ goes to $\infty$ the importance-weighted loss converges to:

$$\begin{aligned}\widehat{err}_{IW}(h) &= \int loss(h(x), y) \frac{P_{test}(x)}{P_{train}(x)} P_{train}(x)\, \mathrm{d}x \\ &= \int loss(h(x), y) P_{test}(x)\, \mathrm{d}x\end{aligned}$$

  which is exactly what we want!
- Thus, if we can compute suitable importance weights, the fact that our training data are biased will not matter (in the limit)

## ALICE and Importance Weighting

- Recall the **oridinary least squares** (OLS) estimate for a linear model $y = X\beta$ is:

$$\hat{\beta} = (X'X)^{-1}X'y$$

- The **importance-weighted Least Squares** estimate minimizes the weighted sum of errors

$$\hat{\beta} = (X'DX)^{-1}X'Dy$$

  where

$$D_{i,j} = \begin{cases} \frac{P_{test}(x_i)}{P_{train}(x_i)} & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

Properties of **Importance-weighted Least Squares**

- It is unbiased, asymptotically, even if the model is misspecified!
- If the model is approximately correct (i.e., $f(x) \approx g(x)$), then the in-model bias is much smaller than the variance (i.e., $B_{in} <<$ Variance)
- If we re-define matrix $L \equiv (X'DX)^{-1}X'D$, then the variance is still

$$Variance = \sigma^2 \mathrm{Tr}(ULL') \propto \mathrm{Tr}(ULL')$$

  - So, we can still use an A-optimal design criterion to reduce variance, and thus the generalization error

## ALICE: Implementation Details

- The traditional approach to finding A-optimal (or D-optimal, etc.) designs is to use **semi-definite programming**
- ALICE doesn't use semi-definite programming
  - Instead, it takes as input a finite set of $k$ probability densities, $\hat{P} = \{P_1, \cdots, P_k\}$
  - It **samples** $n$ instances from each $p \in \hat{P}$ and then
    - Constructs $k$ design matrices, $X_1, \cdots, X_k$, one for each $p \in \hat{P}$
    - Constructs the corresponding $L_1, \cdots, L_k$ matrices
  - The algorithm finds the matrix $L_i$ that minimizes $\mathrm{Tr}(UL_iL_i')$ and then requests for the corresponding set of points
  - It then learns the regression model using importance-weighted least squares

## The ALICE Algorithm

- **Input:** A finite set $\hat{\mathcal{P}}$ of strictly positve probability densities
- Calculate $U$.

$$U_{i,j} = \int \phi_i(x)\phi_j(x)P_{test}(x)\,\mathrm{d}x$$

- For each $p \in \hat{\mathcal{P}}$
  - Create training input points $\{x_i^{(p)}\}_{i=1}^n$ following $p(x)$
  - Calculate $L_W$

$$L = (X'DX)^{-1}X'D$$

- Calculate $J(p)$

$$J(p) = \mathrm{Tr}(ULL')$$

- Choose $\hat{p}$ that minimizes $J$
- Put $x_i = x_i^{(\hat{p})}$ for $i = 1, 2, \ldots, n$.
- Call the oracle and learn the model $\alpha = Ly$, which entails:
  - Observe the training output values $\{y_i\}_{i=1}^n$ at $\{x_i\}_{i=1}^n$
  - Calculate $\hat{\alpha}_W$ by Eq. (9)
- **Output:** $\hat{\alpha}_W$

## ALICE Guarantees

**Error** (assuming that $f(x) = g(x) + \delta r(x)$)

- $\mathbb{E}[Err(x)] = \sigma^2 \mathrm{Tr}(ULL') + O(\delta n^{-1})$
  - Note that $\sigma^2 \mathrm{Tr}(ULL') = O(n^{-1})$, so if $\delta = O(1)$, then the expeted error decreases as $O(n^{-1})$ which is the same asymptotic rate as if the model was specified correctly

## Example Results

- True function: $f(x) = 1 - x + x^2 + \delta x^3 + \epsilon$
  - Assume that $\epsilon \sim \mathcal{N}(0, 0.3)$
- Our assumed model: $g(x) = \alpha_1 - \alpha_2 x + \alpha_3 x^2$
- Compare three different strategies using $k = 10$ labeled instances

| Generalization Error (k = 10) | $\delta = 0$ | $\delta = 0.05$ | $\delta = 0.5$ |
|---|---|---|---|
| ALICE | 1.1 | 1.2 | 4.7 |
| A-optimal design: $\min \mathrm{Tr}\left((X'X)^{-1}\right)$ | 1.1 | 2.0 | 95.5 |
| Random Selection | 2.2 | 2.3 | 7.1 |

## ALICE: Summary

- ALICE actively learns **linear basis function models**, and provides generalization guarantees, **even if the model is misspecified**
- It is intended for a membership query synthesis data access model
- It uses two techniques to (approximately) minimize generalization error
  - **Importance weighting** corrects for in-model bias, resulting in an unbiased estimator
  - The importance weighted **A-optimal design** reduces variance
- It has the same learning rate as passive learning, but produces better models
- **It assumes that we know** $P_{test}$
  - If you don't know $P_{test}$ but do have access to an unlabeled pool of samples from $P_{test}$, the same author does have a closely related algorithm:
    - Pool-based active learning in approximate linear regression

# LapRDD

"Laplacian Regularized D-optimal Design for Active Learning and its Application to Image Retrieval", He (IEEE Trans Image Process, 2009)